

DATA/CS 172: Python for Data Analysis

None

Prof. Travis Mandel

9/25/2024



UNIVERSITY
of HAWAI'I®

HILO

Reminders

- Lab 7 final deadline today
- Lab 8 due Friday

Agenda

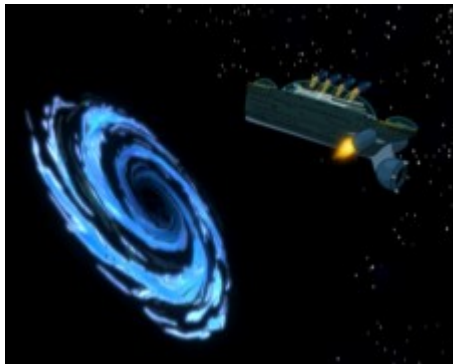
- None

None

- None is a special value in Python
- Any variable can be assigned None
- You will get a `NoneType` exception if you try to do pretty much anything with None
 - Add, divide, iterate, etc.
- To check if a variable is None, use:
 - if `x is None`:
- NOT GOOD TO USE:
 - if `x == None`:
- Also fine to write “if `x is not None`:”



[This Photo](#) by Unknown Author is licensed under [CC BY-NC-ND](#)



None and functions

- One example of where None can be very useful is writing functions
- For example, let's say you are writing a function that computes the average of a list
- If it is passed [6, 7]...
 - ... it should return 6.5
- If it is passed [6]
 - ... it should return 6
- And if it is passed []?
 - It should return None!
- None automatically comes back from functions with no return statement at all (not good style to rely on this - usually a bug/error!)

None is one of the **most important** elements of Python

- And it's particularly hard to grasp the importance for people familiar with C++
- To see why, let's say we want to write a function to get the max element of a list **from scratch** (without using any built-in max functions)...
 - Taking (arg)max is really important in many AI/data science applications



Taking max: without None #1

```
def getMax(values):  
    maxV = 0  
    for value in values:  
        if value > maxV:  
            maxV = value  
  
    return maxV
```

Taking max: without None #2

```
def getMax(values):  
    maxV = 0  
    for value in values:  
        if maxV == 0 or value > maxV:  
            maxV = value  
  
    return maxV
```

-10,-5,0,-20

Taking max: The CS150/C++ way

```
def getMax(values):  
    maxIndex = 0  
    for i in range(len(values)):  
        if values[i] > values[maxIndex]:  
            maxIndex = i  
  
    return values[maxIndex]
```

Works okay in most cases...

Where will it break?

The max() function has the same problem!

Taking max with None

```
def getMax(values):  
    maxV = None  
    for value in values:  
        if maxV is None or value > maxV:  
            maxV = value  
  
    return maxV
```

Finally, a good solution!

Implementing a max & argmax with None

```
def getMaxAndArgMax(values):  
    maxV = None  
    argMax = None  
    for i in range(len(values)):  
        value = values[i]  
        if maxV is None or value > maxV:  
            maxV = value  
            argMax = i  
  
    return (maxV, argMax)
```

Lab out

- Practice with None and functions